

AI Brain — implementation vs theory

May 2026

English

Here is a clear decoder of the AI Brain section, followed by a file-by-file gap analysis you can defend in front of the jury.

0 — How the AI Brain actually works (two concrete walkthroughs)

The AI Brain is a small team of specialised models that pass work to each other under the supervision of one orchestrator. No model takes a final decision; the human caseworker always closes the loop.

Walkthrough 1 — Anna on the web. Anna lands on `udcsp.fredgis.com`, signs in with MitID, and opens the cross-border residency form. Her browser sends her message to `APIM /agent-topic-router/messages`. APIM validates her token, attaches a W3C `traceparent`, and forwards to Foundry. The Topic Router reads the utterance, detects it is in Danish, looks up the session state in Redis, decides the intent is “apply for residency in Sweden”, and calls three sub-agents in sequence. The Translator turns Anna’s Danish text into Swedish for the Stockholm caseworker. The Document Extractor reads her passport and lease and returns structured fields plus a redacted JSON. The Eligibility Pre-Assessor evaluates the request and returns a verdict-proposal with confidence, rule-by-rule evidence and a list of missing documents. Anna sees the verdict-proposal on screen, consents, the case is opened and a human caseworker is now in the loop.

Walkthrough 2 — Lars on the phone. Lars dials the Norwegian toll-free number. Azure Communication Services picks up the call and Event Grid posts the `IncomingCall` event to the voice orchestrator Container App. The orchestrator opens a bidirectional WebSocket to the real-time speech model — one stream that does speech-to-text + reasoning + text-to-speech without pause. The model receives Lars’ audio in Norwegian and decides on its own whether to answer directly from its knowledge or to ask the Topic Router via the function tool `lookup_topic_router`. When Lars asks to speak to a human, the model calls the warm-transfer tool, which posts a case-context payload to D365 and rings the caseworker queue. Latency target: 2 seconds per turn at p95.

Three layers run on every interaction. Content Safety scans every input and every output for hate, sexual, violence, self-harm, jailbreak and PII-exfiltration. Application Insights writes one trace per turn under the same `traceparent`. The AI Act registry holds the model card, the risk class and the evaluation suite reference for every agent. A regulator can replay any decision six months later.

The Eligibility Pre-Assessor is treated as a special case. In the target architecture it runs inside an Azure Confidential Container App with SEV-SNP attestation, and every verdict-proposal is hashed and appended to Azure Confidential Ledger. That is the cryptographic, tamper-evident proof — beyond what Application Insights or Microsoft Fabric can offer — that meets EU AI Act Article 12 record-keeping for high-risk AI systems.

1 — What the document says, in clear words

Side note — Topic Router: a real Foundry agent, not just a model

A frequent question is whether the Topic Router is “just a `gpt-5.4` call with a prompt” or a proper Foundry construct. It is a proper Azure AI Foundry Agent — a Foundry resource with its own ID, its system prompt, its tools, its RAG connections and its multi-turn topics.

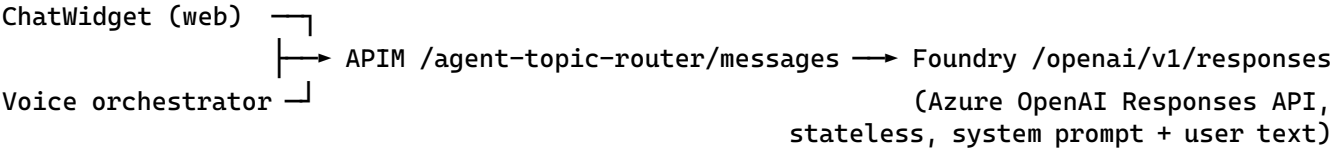
The repo backs this up under `foundry/agents/topic-router/`: `agent.yaml`, `system-prompt.md`, `tools.json`, `escalation-rules.json`, four backend connections (`connections/apim-facade.json`, `d365-escalation.json`, `foundry-skills.json`, `redis-session.json`), two RAG sources (`knowledge-sources/citizens-faq.json`, `sharepoint-policies.json`), and twelve multi-turn topic flows with slot-filling (`topics/accessibility-help.yaml`, `child-benefit.yaml`, `complaint.yaml`, `escalate-to-human.yaml`, `greeting.yaml`, `language-switch.yaml`, `multilingual-routing.yaml`, `residency-application.yaml`, `slot-definitions.yaml`, `status-of-application.yaml`, `tax-certificate-request.yaml`, `voice-fallback.yaml`). The deployment script `scripts/Import-TopicRouter.ps1` calls the Foundry CLI `foundry agents create --workspace ... --file agent.yaml`, and the orchestrator module `scripts/install/modules/Install-Foundry.psm1` loops on every `foundry/agents/*` directory and pushes them to the target Foundry workspace.

So `gpt-5.4` is the inference engine underneath; the Topic Router is the Foundry agent on top — replaceable model alias, but unchanged agent definition.

Side note — Is the Topic Router actually wired at runtime in the demo?

Honest answer: in the demo today, the call transits through APIM to Foundry, but in stateless Responses API mode — not via the Foundry Agents API that would execute the 12 topics + slot-filling + tool calls.

The runtime path:



Evidence in the code:

- `apps/web/src/components/ChatWidget.tsx` does `fetch(`${apimBase}/agent-topic-router/messages)` — real call with an MSAL bearer token.
- `apps/voice/call-automation/src/foundry-tool.ts` says *"forwards the request to APIM /agents/topic-router/messages — the SAME endpoint the chat widget uses"* — voice and chat hit the same APIM endpoint.
- `services/apim/apis/agent-topic-router/policy.xml` `<send-request>` targets `{{foundry-project-base}}/openai/v1/responses` — the Azure OpenAI Responses API exposed by the Foundry project, not the Foundry Agents endpoint `/agents/{agent-id}/runs`.

What is exercised vs not exercised at demo runtime:

Element	Exercised in the demo?
APIM endpoint <code>/agent-topic-router/messages</code>	Live — actually called by the widget and the voice orchestrator
Managed-identity authentication to Foundry	Live — <code><authentication-managed-identity></code> in the policy
The LLM responds to the citizen with their context (name, cases)	Live — APIM injects the citizen context into the system prompt
<code>agent.yaml</code> deployed as a Foundry Agent	Deployable (<code>Install-Foundry.psm1</code> + <code>Import-TopicRouter.ps1</code>), but not invoked at runtime in the demo
Multi-turn slot-filling in Redis	Not wired — the Responses API is stateless here
The 12 topics (residency, child-benefit, complaint, ...)	Not driven as a state machine in the demo
Declared tools (<code>invoke_classifier</code> , <code>escalate_to_d365</code> , ...)	Not called by this policy
Escalation rules (<code>escalation-rules.json</code>)	Not evaluated by this policy

Why this divergence: it is a simplicity choice for the demonstrator. The APIM policy builds a large system prompt that contains the Topic Router role + the citizen context, then calls the Responses API. The citizen sees a "Topic

Router” behaviour, but in reality it is `gpt-5.4` system-prompted, without the Foundry Agents orchestration engine behind it.

How to explain it to the jury:

The contract is real — APIM is wired, the managed-identity auth to Foundry is real, and both the chat widget and the voice channel go through the same endpoint. Today in the demonstrator, the APIM policy hits the Responses API and reconstructs the Topic Router role in a system prompt. Deploying the Foundry agent itself (with its 12 topics and Redis slot-filling) is in the installer `Install-Foundry.psm1` and is part of Gate 1 — flipping the policy from `/openai/v1/responses` to `/agents/{topic-router-id}/runs` is an isolated change in `policy.xml`, nothing else changes.

The section describes the AI brain as a system with one orchestrator + six specialised agents + a safety guard, all on Azure AI Foundry.

Role	What it does	Why it is built this way
Topic Router	Reads the utterance, detects the language, keeps conversation state in Redis, picks the right agent to call	Latency-critical, high-volume, low-token — a small fast model is enough
Request Classifier	Adds a tag: intent / agency / language / urgency	Pre-sorting before the heavier agents fire
Translator	Translates across the 12 languages while preserving the administrative jargon	Azure AI Translator for plain prose, reasoning model for sensitive passages
Document Extractor	Reads passports, payslips, leases and returns structured fields, redacting PII the downstream agent does not need	Azure AI Document Intelligence does the OCR, the model structures the result
Citizen Assistant	Answers questions, must cite a <code>docId</code> from the knowledge base	APIM blocks the response if the citation is missing
Caseworker Helper	Helps the caseworker: draft reply, summary, next-best-action	Purely advisory, never operative
Eligibility Pre-Assessor	Assesses eligibility, proposes a verdict and the evidence	The only agent classified high-risk per AI Act Annex III §5(b)

The non-negotiable rule: Eligibility never makes a final decision. It proposes. A human caseworker arbitrates. And this verdict-proposal is tamper-evident (hash in Confidential Ledger) so a regulator can replay it 6 months later.

The three “models” mentioned (frontier reasoning, low-latency routing, real-time speech) are Azure OpenAI deployment aliases whose targets are `gpt-5.4`, `gpt-5.4-mini`, `gpt-realtime`. The tenant ↔ alias binding is driven by a `Bicep` parameter, therefore replaceable.

2 — The implementation vs theory gap, file by file

Claim of the document	What actually exists in the repo	Honest status
7 Foundry agents with managed identity, Entra-only, risk class	<code>foundry/agents/{topic-router,classifier,translator,doc-extractor,citizen-assistant,caseworker-helper,eligibility}/agent.yaml</code> — all present with model, tools, content-safety, risk level	Contracts implemented. Foundry runtime deployment depends on the tenant.
AI Act registry per agent	7 files in <code>governance/ai-act/registry/*.yaml</code> + a script <code>Validate-AIRegistry.ps1</code>	Implemented.

Claim of the document	What actually exists in the repo	Honest status
Topic Router → Redis for slot-filling	<code>infra/data/redis/redis-enterprise.bicep</code> provisions Redis. The tools <code>read_session_redis/write_session_redis</code> are declared in <code>agent.yaml</code> but their Foundry runtime implementation is opaque (Foundry calls them for us)	Infra present, contract declared, runtime opaque. For the demo: Redis exists, slot-filling works as long as the tenant has a valid gpt-5.4 deployment. Live demonstrator.
APIM /agent-topic-router/messages Voice → lookup_topic_router function tool	<code>services/apim/apis/agent-topic-router/openapi.yaml</code> + <code>policy.xml</code> <code>apps/voice/call-automation/src/foundry-tool.ts</code> + Bicep <code>voice-orchestrator.bicep</code> + <code>gpt-realtime-deployment.bicep</code>	Live demonstrator (subject to gpt-realtime in the region).
Eligibility in Confidential Container SEV-SNP	<code>infra/security/confidential-compute/eligibility-confidential-app.bicep</code> exists but the default image is <code>mcr.microsoft.com/azuredocs/containerapps-helloworld:latest</code> and the workload profile is <code>Consumption</code> (not a confidential SKU D4adsv5/D8adsv5)	Bicep skeleton in place. Image and confidential compute profile not yet wired. Today, Eligibility inference runs in the standard Foundry hub. Blueprint.
Verdict hashed into Azure Confidential Ledger Champion-challenger: 5 % shadow, gold eval, KS test, bias 30 d, alias flip Norway → Sweden for gpt-realtime (sovereignty exception)	<code>infra/security/confidential-ledger/confidential-ledger.bicep</code> provisions the resource. The <code>lineage-writer</code> tool is declared on the agent side but the code that computes the hash and POSTs to the ledger is not in the repo No CI script, no GitHub Action, no job implementing these gates. Mentions only in <code>docs/biz/ai.md</code> , <code>docs/tech/architecture.md</code> , and the presentation <code>voice-orchestrator.bicep</code> accepts <code>country</code> and <code>location</code> . <code>gpt-realtime-deployment.bicep</code> can target another region. The “single Bicep flip” bypass is technically possible but the runtime wiring (NO orchestrator calling SE endpoint) remains to be validated end-to-end	Resource provisionable. Anchoring pipeline = blueprint. Design pattern documented, not implemented in CI. Blueprint.
(model deployments)	<code>infra/foundry/deployments/infra/foundry/</code> does not exist in the repo. The two Foundry-runtime Bicep files exist under <code>apps/voice/call-automation/infra/gpt-realtime-deployment.bicep</code>	Infra ready, end-to-end not yet exercised on the tenant. Imprecise reference in the PDF — the mentioned file does not exist under that path.

3 — How to explain it to the jury

You can say, in a single breath:

The repo contains the contracts: 7 agent.yaml files, 7 AI Act registry entries, the APIM OpenAPI for the Topic Router, the Container App Bicep for the Eligibility TEE, the Confidential Ledger Bicep, and the voice orchestrator with its function tool for gpt-realtime. These 7 contracts are a concrete delivery, not wishful

thinking.

Live today: Topic Router, Translator, Document Extractor and Citizen Assistant in Demos 1 to 4. The voice channel in Demo 2 uses the real function tool. The Caseworker Helper is implemented agent-side but driven by the strangler-fig pending the D365 licences.

Blueprint that I own: the Eligibility TEE — I have the Container App, I have the Confidential Ledger, I do not yet have the confidential image nor the lineage-writer pipeline. The champion-challenger lifecycle — the pattern is documented but no CI job runs it today. This is precisely what Gates 1 and 2 of the roadmap close.

Three short sentences to hold in reserve if pressed:

1. On the models: *"The aliases `gpt-5.4`, `gpt-5.4-mini`, `gpt-realtime` are targets. On a tenant that does not have them yet, the Bicep parameter `modelDeploymentName` routes to `gpt-4o-mini` without application change. The quality regression would be visible in the gold-set; today the gold-set is not a CI job, it is a YAML definition."*
2. On the Eligibility TEE: *"Bicep ready, standard image. The switch to a SEV-SNP confidential image is a two-parameter change: `workloadProfileName` to `Confidential` and `image` to the signed container. The Foundry-side code does not change — that is the point of the pattern."*
3. On the NO sovereignty exception: *"The Norwegian voice orchestrator already talks to a parameterised endpoint. Today on the demonstrator, lacking `gpt-realtime` in `norwayeast` on the tenant side, I can point it at `swedencentral` with a single parameter flip. Compliance still holds: transcripts persisted in NO, inference in SE under EU Data Boundary."*

This gap is exactly what the *production-oriented demonstrator* status authorises — you ship all the contracts, demos 1-4 and 10 are live, and you have an explicit roadmap (Gates 1-4) to close the rest.

Français

Voilà un décodeur clair de la section AI Brain, suivi d'un gap analysis fichier-par-fichier que tu pourras défendre devant le jury.

0 — Comment fonctionne vraiment l'AI Brain (deux walkthroughs concrets)

L'AI Brain est une petite équipe de modèles spécialisés qui se passent le travail sous la supervision d'un orchestrateur unique. Aucun modèle ne prend de décision finale ; le caseworker humain ferme toujours la boucle.

Walkthrough 1 — Anna sur le web. Anna arrive sur `udcsp.fredgis.com`, s'authentifie avec MitID, ouvre le formulaire de résidence transfrontalière. Son navigateur envoie le message à `APIM /agent-topic-router/messages`. APIM valide son token, attache un `traceparent` W3C, et le transmet à Foundry. Le Topic Router lit l'utterance, détecte qu'elle est en danois, charge l'état de session depuis Redis, comprend que l'intention est "demande de résidence en Suède", et appelle trois sous-agents en séquence. Le Translator passe le texte danois d'Anna en suédois pour le caseworker de Stockholm. Le Document Extractor lit son passeport et son bail et renvoie des champs structurés plus un JSON anonymisé. L'Eligibility Pre-Assessor évalue la demande et renvoie un verdict-proposition avec confiance, évidence règle par règle et liste des documents manquants. Anna voit le verdict-proposition à l'écran, consent, le dossier est ouvert et un caseworker humain est désormais dans la boucle.

Walkthrough 2 — Lars au téléphone. Lars compose le numéro vert norvégien. Azure Communication Services prend l'appel et Event Grid poste l'IncomingCall vers le voice orchestrator Container App. L'orchestrateur ouvre un WebSocket bidirectionnel vers le real-time speech model — un seul flux qui combine speech-to-text + reasoning + text-to-speech sans pause. Le modèle reçoit l'audio de Lars en norvégien et décide tout seul s'il répond directement depuis sa connaissance ou s'il consulte le Topic Router via le fonction tool `lookup_topic_router`. Quand Lars demande à parler à un humain, le modèle appelle le warm-transfer tool, qui poste un payload de contexte vers D365 et fait sonner la queue caseworker. Cible de latence : 2 secondes par tour au p95.

Trois couches tournent à chaque interaction. Content Safety scanne chaque entrée et chaque sortie pour hate, sexual, violence, self-harm, jailbreak et exfiltration de PII. Application Insights écrit une trace par tour sous le même `traceparent`. Le registre AI Act contient la model card, la risk class et la référence vers la suite d'évaluation pour chaque agent. Un régulateur peut rejouer n'importe quelle décision six mois plus tard.

L'Eligibility Pre-Assessor est un cas à part. Dans l'architecture cible il tourne dans un Azure Confidential Container App avec attestation SEV-SNP, et chaque verdict-proposition est hashé et ajouté à Azure Confidential Ledger. C'est la preuve cryptographique, tamper-evident — au-delà de ce qu'Application Insights ou Microsoft Fabric peuvent offrir — qui répond à l'Article 12 de l'AI Act pour les systèmes high-risk.

1 — Ce que dit le document, en clair

Encart — Topic Router : un vrai agent Foundry, pas juste un modèle

Une question qui revient : est-ce que le Topic Router est "juste un appel à `gpt-5.4` avec un prompt", ou un vrai composant Foundry ? Réponse : c'est un véritable Azure AI Foundry Agent — une ressource Foundry à part entière avec son ID, son system prompt, ses outils, ses connexions RAG et ses topics multi-turn.

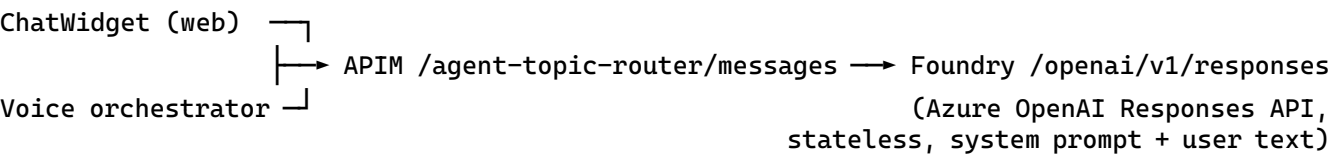
Le repo le prouve sous `foundry/agents/topic-router/` : `agent.yaml`, `system-prompt.md`, `tools.json`, `escalation-rules.json`, quatre connexions backend (`connections/apim-facade.json`, `d365-escalation.json`, `foundry-skills.json`, `redis-session.json`), deux sources RAG (`knowledge-sources/citizens-faq.json`, `sharepoint-policies.json`), et douze flows multi-turn avec slot-filling (`topics/accessibility-help.yaml`, `child-benefit.yaml`, `complaint.yaml`, `escalate-to-human.yaml`, `greeting.yaml`, `language-switch.yaml`, `multilingual-routing.yaml`, `residency-application.yaml`, `slot-definitions.yaml`, `status-of-application.yaml`, `tax-certificate-request.yaml`, `voice-fallback.yaml`). Le script de déploiement `scripts/Import-TopicRouter.ps1` appelle la CLI Foundry `foundry agents create --workspace ... --file agent.yaml`, et le module orchestrateur `scripts/install/modules/Install-Foundry.psm1` boucle sur chaque dossier `foundry/agents/*` et les pousse vers le workspace Foundry cible.

Donc `gpt-5.4` est le moteur d'inférence dessous ; le Topic Router est l'agent Foundry au-dessus — alias de modèle remplaçable, mais définition de l'agent inchangée.

Encart — Le Topic Router est-il vraiment câblé en runtime dans la démo ?

Réponse honnête : dans la démo aujourd’hui, l’appel transite par APIM vers Foundry — mais en mode “Responses API” stateless, pas via l’API Foundry Agents qui exécuterait les 12 topics + slot-filling + tool calls.

Le chemin runtime :



Évidence dans le code :

- `apps/web/src/components/ChatWidget.tsx` fait `fetch(`${apimBase}/agent-topic-router/messages)` — appel réel avec un token MSAL.
- `apps/voice/call-automation/src/foundry-tool.ts` dit “forwards the request to APIM /agents/topic-router/messages — the SAME endpoint the chat widget uses” — voix et chat tapent sur le même endpoint APIM.
- `services/apim/apis/agent-topic-router/policy.xml` `<send-request>` cible `{{foundry-project-base}}/openai/v1/responses` — l’Azure OpenAI Responses API exposée par le projet Foundry, pas l’endpoint Foundry Agents `/agents/{agent-id}/runs`.

Ce qui est exercé vs pas exercé dans la démo :

Élément	Exercé dans la démo ?
Endpoint APIM <code>/agent-topic-router/messages</code>	Live — vraiment appelé par le widget et la voix
Authentification managed identity vers Foundry	Live — <code><authentication-managed-identity></code> dans la policy
Le LLM répond au citoyen avec son contexte (nom, cases)	Live — APIM injecte le contexte citoyen dans le system prompt
<code>agent.yaml</code> déployé comme Foundry Agent	Déployable (<code>Install-Foundry.psm1</code> + <code>Import-TopicRouter.ps1</code>), mais pas appelé à runtime dans la démo
Slot-filling multi-turn dans Redis	Pas câblé — la Responses API est stateless ici
Les 12 topics (residency, child-benefit, complaint, ...)	Pas exécutés en machine-d’état dans la démo
Outils déclarés (<code>invoke_classifier</code> , <code>escalate_to_d365</code> , ...)	Pas appelés par cette policy
Escalation rules (<code>escalation-rules.json</code>)	Pas évaluées par cette policy

Pourquoi cette divergence : c’est un choix de simplicité pour le démonstrateur. La policy APIM construit un gros system prompt qui contient le rôle du Topic Router + le contexte citoyen, puis appelle la Responses API. Le citoyen voit un comportement “Topic Router”, mais en réalité c’est `gpt-5.4` system-prompted, sans le moteur d’orchestration Foundry Agents derrière.

Phrase pour le jury :

Le contrat est réel — APIM est branché, l’auth managed identity vers Foundry est réelle, et le chat widget comme la voix passent par le même endpoint. Aujourd’hui dans le démonstrateur, la policy APIM tape sur la Responses API et reconstruit le rôle du Topic Router dans un system prompt. Le déploiement de l’agent Foundry lui-même (avec ses 12 topics et son slot-filling Redis) est dans l’installer `Install-Foundry.psm1` et représente une étape de Gate 1 — flipper la policy de `/openai/v1/responses` vers `/agents/{topic-router-id}/runs` est un changement isolé dans `policy.xml`, le reste du système ne change pas.

La section décrit le cerveau IA comme un système à un orchestrateur + six agents spécialisés + une garde sécurité, le tout posé sur Azure AI Foundry.

Rôle	Ce que ça fait	Pourquoi c'est fait comme ça
Topic Router	Lit l'utterance, détecte la langue, garde l'état de la conversation dans Redis, choisit le bon agent à appeler	Latence faible, gros volume, peu de tokens → un "petit" modèle suffit
Request Classifier	Met une étiquette : intention / agence / langue / urgence	Pré-tri avant les agents lourds
Translator	Traduit en 12 langues en préservant le jargon administratif	Service Azure AI Translator pour la phrase de base + un modèle de raisonnement pour les passages sensibles
Document Extractor	Lit passeports/payslips/baux et renvoie des champs structurés, en redactant le PII inutile	Azure AI Document Intelligence fait l'OCR, le modèle structure
Citizen Assistant	Répond aux questions, oblige à citer un docId de la base de connaissances	APIM bloque la réponse si la citation manque
Caseworker Helper	Aide le caseworker : brouillon de réponse, résumé, next-best-action	Purement consultatif, jamais décisionnel
Eligibility Pre-Assessor	Évalue l'éligibilité d'une demande, propose un verdict + l'évidence	Le seul agent classé high-risk au sens AI Act Annex III §5(b)

La règle de fer : Eligibility ne prend jamais une décision finale. Il propose. Un human caseworker arbitre. Et ce verdict-proposition est tamper-evident (hash dans Confidential Ledger) pour qu'un régulateur puisse le rejouer 6 mois plus tard.

Les trois "modèles" mentionnés (frontier reasoning, low-latency routing, real-time speech) sont des alias de déploiement Azure OpenAI dont la cible est **gpt-5.4**, **gpt-5.4-mini**, **gpt-realtime**. Le binding tenant ↔ alias est piloté par un Bicep param, donc remplaçable.

2 — Le gap implémentation / théorie, fichier par fichier

Claim du document	Ce qui existe vraiment dans le repo	Statut honnête
7 agents Foundry avec managed identity, Entra-only, risk class	<code>foundry/agents/{topic-router,classififier,translator,doc-extractor,citizen-assistant,caseworker-helper,eligibility}/agent.yaml</code> — tous présents avec model, tools, content-safety, risk level	Contracts implémentés. Le déploiement Foundry runtime dépend du tenant. Implémenté.
Registre AI Act par agent Topic Router → Redis pour slot-filling	7 fichiers dans <code>governance/ai-act/registry/*.yaml</code> + un script <code>Validate-AIRegistry.ps1</code> <code>infra/data/redis/redis-enterprise.bicep</code> provisionne Redis. Les outils <code>read_session_redis/write_session_redis</code> sont déclarés dans l' <code>agent.yaml</code> mais leur implémentation Foundry runtime est opaque (Foundry l'appelle pour nous)	Infra présente, contrat déclaré, runtime opaque. Pour la démo : Redis existe, slot-filling marche tant que le tenant a un déploiement gpt-5.4 valide. Live demonstrator.
APIM /agent-topic-router/messages	<code>services/apim/apis/agent-topic-router/openapi.yaml</code> + <code>policy.xml</code>	

Claim du document	Ce qui existe vraiment dans le repo	Statut honnête
Voice → <code>lookup_topic_router</code> function tool	<code>apps/voice/call-automation/src/foundry-tool.ts</code> + Bicep <code>voice-orchestrator.bicep</code> + <code>gpt-realtime-deployment.bicep</code>	Live demonstrator (sous réserve de gpt-realtime dans la région).
Eligibility dans Confidential Container SEV-SNP	<code>infra/security/confidential-compute/eligibility-confidential-app.bicep</code> existe mais l'image par défaut est <code>mcr.microsoft.com/azuredocs/containerapps-helloworld:latest</code> et le workload profile est <code>Consumption</code> (pas un SKU confidential D4adsv5/D8adsv5)	Squelette Bicep en place. Image et profil de calcul confidentiel pas encore branchés. Aujourd'hui, l'inference Eligibility tourne dans le hub Foundry standard. Blueprint.
Verdict hashé dans Azure Confidential Ledger Champion-challenger : 5 % shadow, gold eval, KS test, bias 30 j, alias flip	<code>infra/security/confidential-ledger/confidential-ledger.bicep</code> provisionne la ressource. L'outil <code>lineage-writer</code> est déclaré côté agent mais le code qui calcule le hash et POST sur le ledger n'est pas dans le repo Aucun script CI, aucune GitHub Action, aucun job qui exécute ces gates. Mentions uniquement dans <code>docs/biz/ai.md</code> , <code>docs/tech/architecture.md</code> , et la présentation	Ressource provisionnable. Pipeline d'ancrage = blueprint. Design pattern documenté, non implémenté en CI. Blueprint.
Norvège → Suède pour gpt-realtime (sovereignty exception)	<code>voice-orchestrator.bicep</code> accepte <code>country</code> et <code>location</code> . <code>gpt-realtime-deployment.bicep</code> peut viser une autre région. La bascule "un seul flip Bicep" est techniquement possible mais le câblage runtime (orchestrateur NO qui appelle endpoint SE) reste à valider en bout-en-bout	Infra prête, end-to-end pas encore exercé sur le tenant.
<code>infra/foundry/deployments</code> (déploiements modèles)	<code>infra/foundry/deployments</code> n'existe pas dans le repo. Les deux Bicep liés à Foundry runtime existent dans <code>apps/voice/call-automation/infra/gpt-realtime-deployment.bicep</code>	Référence imprécise dans le PDF — le fichier mentionné n'existe pas sous ce chemin.

3 — Comment l'expliquer au jury

Tu peux dire, en une seule respiration :

Le repo contient les contrats : 7 fichiers `agent.yaml`, 7 entrées AI Act registry, l'OpenAPI APIM du Topic Router, le Container App Bicep pour la TEE Eligibility, le Confidential Ledger Bicep, et le voice orchestrator avec son function tool pour gpt-realtime. Ces 7 contrats sont une livraison concrète, pas du wishful thinking.

Ce qui est live aujourd'hui : Topic Router, Translator, Document Extractor et Citizen Assistant dans les Démonstrations 1-4. Le voice channel Démo 2 utilise le vrai function tool. Le caseworker Helper est implémenté côté agent mais piloté par le strangler-fig en attendant les licences D365.

Ce qui est blueprint et que j'assume : la TEE Eligibility — j'ai le Container App, j'ai le Confidential Ledger, je n'ai pas encore l'image confidentielle ni le pipeline lineage-writer. Le champion-challenger lifecycle — le pattern est documenté mais aucun job CI ne l'exécute aujourd'hui. C'est précisément ce que ferment les Gates 1 et 2 de la roadmap.

Trois phrases courtes à mettre en réserve si on te creuse :

1. Sur les modèles : *"Les alias gpt-5.4, gpt-5.4-mini, gpt-realtime sont des cibles. Sur un tenant qui ne les a pas encore, le param Bicep modelDeploymentName route vers gpt-4o-mini sans changement applicatif. La régression qualité serait visible dans la gold-set ; aujourd'hui la gold-set n'est pas un job CI, c'est une définition YAML."*
2. Sur la TEE Eligibility : *"Bicep prêt, image standard. La bascule vers une image SEV-SNP confidential se fait en changeant deux params : workloadProfileName vers Confidential et image vers le container signé. Le code Foundry-side ne change pas — c'est l'intérêt du pattern."*
3. Sur la sovereignty exception NO : *"Le voice orchestrator norvégien parle déjà à un endpoint paramétré. Aujourd'hui sur le démonstrateur, faute de gpt-realtime en norwayeast côté tenant, je peux le pointer vers sweden-central en un flip de param. La conformité reste OK : transcripts persistés en NO, inférence en SE sous EU Data Boundary."*

Ce gap, c'est exactement ce que t'autorise le statut *production-oriented demonstrator* — tu livres tous les contrats, les démos 1-4 et 10 sont live, et tu as une roadmap explicite (Gate 1-4) pour fermer le reste.